# Minnal-Q

## മിന്നൽ

“Lightning-fast quantum routing for life-critical emergencies”

Severity- and Traffic-Aware Multi-Stop Ambulance Routing with QAOA

Combined project concept · QFF26 Kochi Hack · 1 November 2026

| Item | Detail |
|---|---|
| Name | **Minnal** is Malayalam for lightning: pure energy, arriving in an instant. **Q** stands for the quantum (QAOA) engine. The name echoes Kerala's own superhero, Minnal Murali, and the idea that the right route has to reach a patient at the speed of lightning. |
| Challenge | **Challenge 1: Emergency Vehicle Routing** (Quantum Optimization · Smart Mobility) |
| One-line pitch | When one ambulance must reach several patients, Minnal-Q picks the visiting order that gets the most critical patients first under live traffic. The route is found with QAOA on a QUBO in Qiskit and checked against exact and greedy classical solvers. |
| SDG relevance | SDG 3 (Good Health: faster critical care) · SDG 11 (Sustainable Cities: resilient urban mobility) · disaster response (e.g. Kerala flood evacuation) |
| Built from | Patterns from three reference projects (credited in Section 9). All code is written fresh on the day, as the originality rule requires. |
| Stack | Python 3 · Qiskit 2.6 · qiskit-aer 0.18 · qiskit-ibm-runtime (optional) · NumPy/SciPy · Streamlit · Plotly |

## 1. Problem statement

In a mass-casualty event such as a bus accident on NH-66, monsoon flooding or a building collapse, one ambulance often has to collect patients from several pickup points before reaching a hospital. Dispatchers usually take the nearest point first. That keeps total driving short, but it can leave a critically injured patient waiting while minor cases are picked up. Kochi's traffic varies sharply by time of day (Edappally, Vyttila, Kaloor, MG Road), so the best order at 9 a.m. can differ from the best order at 2 p.m.

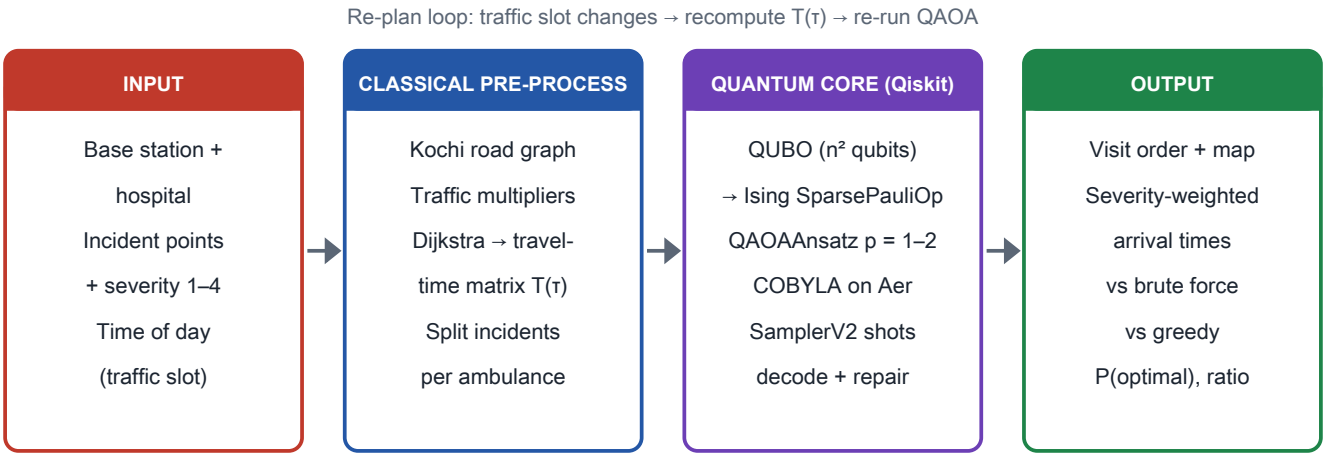
The people affected are patients whose survival depends on time to treatment (cardiac arrest, trauma, stroke), plus ambulance crews and hospital emergency departments. **What we optimize:** the order in which an ambulance visits n incident points, starting from its base and ending at a hospital. The goal is to minimize traffic-adjusted travel time while strongly preferring early visits to high-severity patients. Choosing a visiting order is a permutation problem. The number of possible routes grows as n!, and adding severity and time-dependent travel times makes simple nearest-first rules unreliable. That makes it a natural fit for a QUBO formulation solved with QAOA.

## 2. The combining idea: what we take from each reference

None of the 14 projects we reviewed fully covers Challenge 1 (ambulances + multi-stop routes + distance + traffic). The three strongest each cover part of it. Minnal-Q combines those parts and adds the missing piece, time-dependent traffic.

| Reference | What it does well | Pattern we reuse | What Minnal-Q adds |
|---|---|---|---|
| **QuRoute** (QuHack 2026) Open repo ↗ | Multi-stop TSP with QAOA in Qiskit; brute-force and greedy baselines; map demo | Position-encoded route QUBO; three-way comparison of the routes on a map | Ambulance framing; fixed base → hospital endpoints; traffic-dependent costs; QAOAAnsatz on Qiskit 2.6 instead of the unmaintained qiskit_algorithms; no hidden non-quantum fallback |
| **Severity-Aware Dispatch** (Qiskit Fall Fest 2026) Open repo ↗ | Clean QUBO → Ising → QAOA chain; severity weights; CVaR; penalty sweep; verified against an exact solver | Exponential severity weights (1, 2, 4, 8); QUBO/Ising equivalence test; CVaR objective; penalty-tuning experiment | Severity applied to visiting order (routing), not only to which ambulance goes where |
| **PairCover-Q / Q-Rescue** (Q-HACK India 2026) Open repo ↗ | Real IBM hardware runs; strong tests; honest evidence tables; polished Streamlit app | Transparency table (implemented / simulated / measured); a small hardware run; evidence page in the demo | Same honesty standard applied to a routing problem; an optional 9-qubit IBM run as a bonus |
| **New in Minnal-Q** | — | — | **Time-of-day traffic multipliers** on a Kochi junction graph, a severity-vs-time trade-off slider, and re-planning when the traffic slot changes |

## 3. Proposed solution: Input → Processing → Output




- **Input:** ambulance base, destination hospital, 3–4 incident points with severity 1–4 (minor → critical), and the departure time slot.
- **Classical pre-processing:** a hand-built graph of about 10 Kochi junctions and hospitals. Each road's base travel time is multiplied by a congestion factor for the time slot (e.g. 1.8× on Edappally–Vyttila at peak). Dijkstra's algorithm turns this into an all-pairs travel-time matrix T(τ). With several ambulances, incidents are first split between them classically, and each route is then solved separately.
- **Quantum core:** build the QUBO, convert it to an Ising Hamiltonian, optimize QAOA parameters on the Aer simulator, sample bitstrings, decode them into a visiting order, and keep the best feasible route.
- **Output:** the route drawn on a map; per-patient arrival times; total and severity-weighted time; a comparison with the exact optimum (brute force) and with nearest-first greedy; QAOA quality metrics.

# 4. Quantum component (mandatory section)

## 4.1 Why QAOA fits

Choosing a visiting order is a binary combinatorial problem: each patient goes in exactly one route position, and the travel cost depends on which pairs of patients are consecutive. That pairwise dependence is exactly the quadratic structure a QUBO captures and QAOA is built to optimize. The quantum part therefore solves the routing decision itself; it is not a decorative circuit. This addresses the rule that "simply adding a Hadamard gate or a random quantum circuit does not qualify".

## 4.2 Problem → qubits mapping

Binary variable $x_{i,p} = 1$ if incident i is visited at position p (i, p ∈ {1…n}). The base S and hospital H are fixed endpoints, so they need no qubits. Qubit count = $n^2$: **n = 3 → 9 qubits** (6 possible routes), **n = 4 → 16 qubits** (24 routes). Both sizes run quickly on the Aer simulator.

## 4.3 QUBO cost function

$$H(x) = \mathbf{A}\cdot\Sigma_i(1 - \Sigma_p\, x_{i,p})^2 + \mathbf{A}\cdot\Sigma_p(1 - \Sigma_i\, x_{i,p})^2 \text{ (each patient once, each slot once)}$$
$$+ \Sigma_i\, T_{S,i}(\tau)\cdot x_{i,1} + \Sigma_{i\neq j}\,\Sigma_{p<n}\, T_{i,j}(\tau)\cdot x_{i,p}x_{j,p+1} + \Sigma_i\, T_{i,H}(\tau)\cdot x_{i,n} \text{ (traffic-adjusted travel time)}$$
$$+ \boldsymbol{\lambda}\cdot\Sigma_i\,\Sigma_p\, w_i\cdot(p - 1)\cdot x_{i,p} \text{ (severity: critical patients pushed earlier)}$$

- $w_i \in \{1, 2, 4, 8\}$ for severity 1–4 (exponential, following Severity-Aware Dispatch).
- λ is the severity-vs-time trade-off, exposed as a slider in the demo. λ = 0 gives a plain fastest route.
- A (constraint penalty) is set above the largest possible route cost, then tuned with a penalty sweep. We test that the lowest-energy QUBO state is always a valid route equal to the brute-force optimum.
- **Honest modelling note:** exact severity-weighted arrival time would need terms of degree higher than two, so the QUBO uses a position penalty as a quadratic proxy. Results always report the exact severity-weighted arrival time, recomputed classically for each decoded route.

## 4.4 Circuit and algorithm (Qiskit)

| Step | Qiskit implementation |
|---|---|
| QUBO → Ising | Substitute x = (1 − Z)/2 and build a SparsePauliOp of Z and ZZ terms plus an offset. A unit test checks that the Ising energy equals the QUBO energy for every bitstring. |
| Ansatz | qiskit.circuit.library.QAOAAnsatz(cost_operator, reps=p), p = 1 and 2: start in the uniform superposition $H^{n^2}\|0\ldots0\rangle$, apply alternating cost layers $e^{-i\gamma H_C}$ and mixer layers $e^{-i\beta\Sigma X}$. |
| Training | Aer EstimatorV2 (or StatevectorEstimator) computes the expected cost $E[H_C]$; SciPy COBYLA with 3–5 random restarts; the p = 2 run is warm-started from the best p = 1 angles. A CVaR (α = 0.2) objective is an option. |
| Sampling | Aer SamplerV2, 4,096 shots → bitstrings → decode the n×n position matrix → feasibility check → the lowest-cost valid route wins. Invalid samples are counted and reported, not hidden. |
| Backend | **Primary: qiskit_aer.AerSimulator** (named in the README). Optional bonus: one 9-qubit instance transpiled with generate_preset_pass_manager and run on an IBM backend through QiskitRuntimeService / SamplerV2, with the job ID recorded. |
| Hybrid loop | Classical: graph, Dijkstra, splitting incidents between ambulances, parameter optimizer, decoding. Quantum: evaluating and sampling the QAOA state. The two alternate until the optimizer converges. |

# 5. Classical baselines and evaluation

| Solver | Purpose |
|---|---|
| Brute force (all n! orders) | Exact ground truth for every instance; also validates the QUBO |
| Nearest-first greedy | What dispatchers do today; shows the failure case where a critical patient waits |
| Time-only optimum (λ = 0) | Shows the cost of adding severity: a little extra driving, much faster care for critical patients |
| QAOA p = 1, p = 2 (Aer) | The quantum solver under test |

**Metrics reported:** total route time; exact severity-weighted arrival time $\Sigma w_i \cdot t_i$; critical-patient arrival time; approximation ratio vs brute force; probability of sampling the optimal route; share of valid samples; circuit depth and two-qubit gate count after transpiling. All numbers are measured on the day. None are assumed in this document.

## 6. Demo plan (Streamlit, live at the showcase)

- **Scenario tab:** Kochi map with base, hospital and incident pins; severity picker; time-slot selector (peak / off-peak).
- **Route tab:** three routes overlaid (greedy, time-only, Minnal-Q) with a per-patient arrival table, headed by the critical patient's arrival time.
- **Quantum lab tab:** QAOA circuit diagram, convergence curve, histogram of sampled routes with the optimum highlighted, and p = 1 vs p = 2 results.
- **Evidence tab:** transparency table, QUBO/Ising test results, penalty sweep, and the IBM job ID if a hardware run was done.
- **Demo story (≈ 3 min):** (1) greedy collects the minor case first and the critical patient waits; (2) switch the slot to peak traffic and watch the route change; (3) Minnal-Q's route matches the brute-force optimum; (4) the honesty slide.

## 7. Build plan for today

| Time | Milestone | Done when |
|---|---|---|
| 10:00–10:45 | Kochi graph + traffic multipliers + Dijkstra matrix; brute-force and greedy solvers | Prints all routes and costs for a 3-patient scenario |
| 10:45–12:00 | QUBO builder, QUBO → Ising converter, equivalence tests | QUBO minimum = brute-force optimum; tests pass |
| 12:00–13:30 | QAOAAnsatz + COBYLA + SamplerV2 on Aer; decoding; p = 1/2; penalty sweep | QAOA returns a valid route; metrics CSV saved |
| 13:30–14:00 | Lunch | — |
| 14:00–15:15 | Streamlit app (4 tabs), map, charts | Full flow runs from one command |
| 15:15–16:00 | README, architecture diagram, screenshots; optional 9-qubit IBM run | A fresh clone runs by following the README |
| 16:00–16:30 | Code freeze, rehearse demo, submit on the portal | Submitted before the deadline |

### Suggested roles (team of up to 4)

| Role | Owns |
|---|---|
| Quantum lead | QUBO, Ising, QAOA, experiments, optional IBM run |
| Classical / data lead | Kochi graph, traffic model, Dijkstra, baselines, metrics, tests |
| Demo lead | Streamlit app, map, charts, demo script |
| Docs lead | README, problem statement, diagram, transparency table, presenting |

**Dependencies to add today:** pip install streamlit plotly networkx (Qiskit, Aer, Runtime, Matplotlib and pylatexenc are already installed).

## 8. Requirements compliance checklist

| Kochi requirement | How Minnal-Q meets it |
|---|---|
| 1. Project information | Team, project, challenge and members at the top of the README |
| 2. Problem statement (150–300 words) | Section 1 of this document, reused in the README |
| 3. Solution + Input → Processing → Output | Section 3 with the architecture diagram |
| 4. Quantum component (mandatory) | Section 4: why QAOA fits, qubit mapping, QUBO, circuit, its role in the system |
| 5. Implementation & reproducibility | .py modules and a notebook, requirements.txt, README run steps, fixed random seeds |
| 6. Qiskit usage + backend named | QAOAAnsatz, SparsePauliOp, Aer EstimatorV2/SamplerV2; backend: AerSimulator |
| Rule: original work | Coded on the day; references credited in Section 9 |
| Rule: transparency | Table in the README: implemented / simulated / measured / incomplete |
| Rule: authorized data | Synthetic incidents and travel times on public place names; no patient data |
| Extras recommended | Architecture diagram, screenshots, classical comparison, future work |

### Judging alignment (Phase 2)

| Criterion (weight) | Where we score |
|---|---|
| MVP functionality (25%) | End-to-end app: scenario → QAOA route → comparison, in one command |
| Quantum implementation (25%) | A real QUBO with tested Ising equivalence, p = 1/2, CVaR, measured approximation ratios |
| Technical implementation (20%) | Clean modules, unit tests, reproducible seeds, optional hardware run |

| Criterion (weight) | Where we score |
|---|---|
| Innovation & impact (15%) | Severity + time-of-day traffic in routing, which none of the 14 reference projects combine; Kochi/Kerala context |
| Demo & presentation (15%) | Three-minute story built on the failure case of nearest-first dispatch |

## 9. Transparency, limitations and credits

- **Simulated:** all main QAOA results come from AerSimulator. The traffic multipliers and incident locations are synthetic but realistic; place names are real Kochi locations.
- **Scale:** up to 4 patients per ambulance (16 qubits). Larger cases are split classically. **No quantum advantage is claimed**; the page states this is not expected.
- **Proxy:** the QUBO uses a position-based severity penalty; the exact severity-weighted metric is computed classically.
- **Credits:** patterns only, no code copied (see the reference repositories table below). Qiskit and Qiskit Aer (IBM, Apache 2.0).

### Reference repositories (combined in Minnal-Q)

| Project | Event | Repository link (clickable) | What we took |
|---|---|---|---|
| **QuRoute** | QuHack 2026 | https://github.com/AnuragKannojiya/quroute | Multi-stop route QUBO, brute-force + greedy baselines, map demo |
| **Severity-Aware Emergency Dispatch Using QAOA** | IBM Qiskit Fall Fest 2026, RV University | https://github.com/shrisha77-boop/ Severity-Aware-Emergency-Dispatch-Using-QAOA | Severity weights, QUBO → Ising check, CVaR, penalty sweep |
| **PairCover-Q / Q-Rescue** | Q-HACK INDIA 2026 | https://github.com/gautham-code505/PairCover-Q- | Transparency table, IBM hardware run, evidence page |

### Future work

- Real Kochi road network from OpenStreetMap (OSMnx) and a live traffic API.
- Kerala-flood scenario: boats and ambulances with blocked roads as hard constraints.
- Hospital capability and bed capacity in the destination choice.
- Warm-start QAOA and error mitigation on IBM hardware for larger instances.